IIT JODHPUR

Feb'26 Minor Examination: EEL3090 Embedded Systems $\boxed{Set - A}$

Guidelines (Total time: 120 minutes, Maximum Marks: 25):

- Please read question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume and state that in your answer.
- **Please mention Set Number in your answer-sheet.**

---

1. You are working as a system architect at *MarwarEmbedded*, where you are redesigning a real-time text-matching embedded platform intended for secure industrial terminals. The system receives 8-bit ASCII characters from a keyboard at unpredictable intervals. The system must perform real-time pattern matching against a stored reference string (maximum length = 64 characters) and transmit only the matched characters to a serial LCD terminal with UART-style transfer that follows 8 data bits, no parity, 1 stop bit (8N1 format) and baud-rate deployed is known as 9600. The microcontroller runs at a clock frequency of 15.97 MHz and includes these components: UART peripheral, Interrupt controller, DMA controller, Timer modules, GPIO, and Internal SRAM. Your system must satisfy the following additional constraints: i) The keyboard may generate bursts of characters at up to 5 kHz. ii) The LCD cannot tolerate buffer overrun, and iii) The CPU must remain in low-power sleep mode whenever possible
   - Derive the UART baud rate divisor required to generate 9600 bps from a 15.97 MHz clock ? [1]
   - Considering 8N1 transmission format, calculate the maximum number of characters per second that can be transmitted to the LCD? [1]
   - Determine whether the system can safely handle a 5 kHz burst from the keyboard without data loss? If answer to above is NO, suggest one remedy? If answer is yes, write N/A? [1 + 1]
   - Draw a detailed block-level architecture of the complete system, clearly identifying the internal microcontroller components used. [1]
2. Convert below legacy ARM assembly into equivalent C/C++ sub-routine (lr is link register)? [3]

```
        PUSH   {r4, r5, lr}
        MOV    r2, #0
        MOV    r3, #0
loop:
        CMP    r3, r1
        BGE    done
        LDR    r4, [r0, r3, LSL #2]
        CMP    r4, #0
        BLE    skip
        ADD    r2, r2, r4
skip:
        ADD    r3, r3, #1
        B      loop
done:
        MOV    r0, r2
        POP    {r4, r5, lr}
        BX     lr
```

3. You are hired as an embedded systems consultant at a defense startup developing a battlefield electronic intelligence (ELINT) system. The objective is to detect, analyze, and intercept enemy radar signals near the India-Pakistan border region. This system passively receives high-frequency square-wave radar pulses emitted by an enemy radar installation. These intercepted pulses are first conditioned using an RF front-end and comparator circuit to convert them into clean digital pulses suitable for a microcontroller input pin. You decide to use an STM32 Nucleo board operating at 16 MHz to measure the pulse width and repetition characteristics of the incoming radar signal. The goal is to: i) measure the pulse width

(ON time) of the radar signal, ii) determine its pulse repetition interval (PRI), and iii) estimate radar characteristics based on timing analysis Explain (with relevant diagrams), step-by-step, how you would configure and utilize the microcontroller's hardware timer in input capture mode to (assuming that the intercepted radar pulse is periodic): for identifying (or classifying) the enemy radar system? [3]

4. For the purpose of interrupt generation using hardware timers, you are approached by your friend in *TexasInstruments* company. (a) For a 48 MHz clock, calculate prescaler and compare value required to generate a 100 ms interrupt using the 16-bit timer. (b) Can this 16-bit hardware timer directly generate a 30-second delay? Show calculation and explain your approach if not. [1.5 + 1.5]

5. You are designing a battery health monitoring unit for a satellite onboard computer for ISRO. The system uses an internal ADC to monitor battery voltage via an analog input pin. The microcontroller has: 12-bit ADC, One analog input channel connected to battery voltage divider, GPIO pins configurable as digital input/output, ADC End-Of-Conversion (EOC) flag, ADC conversion complete interrupt and 48 MHz system clock. The ADC conversion time is 15 μs. The system does the following: start an ADC conversion every 5 ms, detect when the conversion is complete, toggle a GPIO debug pin HIGH when conversion starts, toggle the GPIO LOW when conversion result is read.

   There are two methods to achieve the above sequence of tasks: either through continuous monitoring of respective flags or through interrupts. Draw the software/application flow (main loop and functions) for both these methods? Mention the merits/demerits of both these methods? [2 + 2]

6. You are hired as an embedded systems engineer in a safety-critical industrial automation company. You are asked to design a multi-level alarm generation system using an STM32 microcontroller. This kit has the following specifications: a) The STM32 system clock (HCLK) is configured to 84 MHz. b) SysTick is a 24-bit down counter, and it can use either: i) HCLK (84 MHz), or ii) HCLK/8 as its clock source. The system designed by you must generate the following:
   a) A Level-1 Warning Alarm every 250 ms, b) A Level-2 Critical Alarm every 2 seconds
   Determine whether it is possible to directly generate a 2-second interrupt using SysTick without any involvement from software (in the form of software counters etc.)?
   If possible, calculate: required reload value? If not, explain why ? [2 + 1.5]

7. You are hired as an embedded firmware engineer in a defense-grade hardware startup and are required to design a secure multi-mode control interface using bare-metal register-level programming on an STM32 microcontroller (no HAL, no CubeMX, and no driver libraries allowed). The system runs at an HCLK frequency of 80 MHz and must use only GPIO peripherals.
   Three push buttons are connected to GPIOA pins PA0, PA1, and PA2 in pull-down configuration, and two LEDs are connected to GPIOB pins PB5 and PB6 configured as push-pull outputs. The firmware must implement three operating modes using only polling-based logic inside the main loop. In Normal Mode, pressing PA0 toggles LED1 and pressing PA1 toggles LED2. In Secure Mode, PA0 toggles LED1 only if PA1 is pressed within a 200 ms window, while PA2 immediately turns off both LEDs and invalid sequences are ignored. In Lockdown Mode, which is activated when all three buttons are pressed simultaneously, the LEDs must blink alternately at approximately 5 Hz, and the system should exit this mode only if PA2 is continuously held for more than 3 seconds.
   You must describe, in detail, how you would configure the required RCC and GPIO registers, implement reliable software debouncing using only polling, detect simultaneous button presses, generate approximate timing delays using calibrated software loops based on the 80 MHz clock, and structure the main control loop as a finite-state machine to ensure deterministic behavior without using interrupts or hardware timers. Additionally, explain how you would prevent race conditions in shared variables and ensure stable GPIO state transitions in a purely polling-based bare-metal implementation. (Note: polling means continuous monitor/checking of flag values/flag registers etc.) [3.5]
